



Edited with the trial version of
Foxit Advanced PDF Editor

To remove this notice, visit:

www.foxitsoftware.com/shopping

Database Manipulation with dbplyr

Nội dung

Giới thiệu dbplyr package

Kết nối đến database

Các loại phép kết

Giới thiệu dbplyr package

Các thao tác cơ bản với dplyr

- ▶ `select()`: chọn dòng
- ▶ `filter()`: chọn dòng dùng điều kiện trên giá trị
- ▶ `mutate()`: tạo biến mới là một hàm trên các biến đã có
- ▶ `summarise()`: tổng hợp kết quả từ nhiều dòng
- ▶ `group_by()`: nhóm các dòng theo giá trị
- ▶ `arrange()`: sắp xếp dòng

dbplyr package

- ▶ dplyr có thể làm việc với in-memory data ở dạng data frame cũng như on-disk data được lưu trong database.
- ▶ Để sử dụng database với dplyr với database ta cần cài dbplyr package.
 - ▶ DBI package sẽ được cài khi ta cài dplyr package
 - ▶ DBI package cung cấp một giao diện chung cho phép dplyr làm việc với nhiều database khác nhau với cùng một mã.

```
install.packages("dbplyr")
```

- ▶ Ta cần cài một backend cụ thể để giao tiếp với mỗi loại database khác nhau.

Specific backends

- ▶ Các backend phổ biến
 - ▶ RMariaDB kết nối đến MySQL và MariaDB
 - ▶ RPostgres kết nối đến Postgres và Redshift.
 - ▶ RSQLite kết nối đến SQLite database.
 - ▶ odbc: kết nối đến nhiều commercial databases qua open database connectivity protocol.
 - ▶ bigrquery: kết nối đến Google's BigQuery
- ▶ Phần tiếp theo sẽ dùng RSQLite backend, vốn tự động được cài khi ta cài dbplyr package.
 - ▶ SQLite được nhúng (embedded) trong một R package.
 - ▶ Ta không cần thiết lập một database server như các database khác.

Kết nối đến database

Kết nối đến database (1)

- ▶ Để dùng dplyr, ta cần kết nối nó dùng DBI::dbConnect()

```
library(dplyr)
con <- DBI::dbConnect(RSQLite::SQLite(), dbname = ":memory:")
```

- ▶ Đối số của DBI::dbConnect() khác nhau tùy vào loại database, nhưng tham số đầu tiên là database backend.
 - ▶ RSQLite::SQLite() cho RSQLite.
 - ▶ RPostgres::Postgres() cho RPostgres
 - ▶ RMariaDB::MariaDB() cho RMariaDB
 - ▶ odbc::odbc() cho odbc
 - ▶ bigrquery::bigquery() cho BigQuery
- ▶ Đối số thứ hai là đường dẫn đến database. Ở đây, chuỗi ":memory:" mô tả một temporary in-memory database.

Kết nối đến database (2)

- ▶ Nếu database nằm ở một server khác, mã kết nối sẽ tương tự như sau:

```
con <- DBI::dbConnect(RPostgres::RPostgres(),  
  host = "database.rstudio.com",  
  user = "user_name",  
  password = rstudioapi::askForPassword("database_password")  
)
```

- ▶ Tuy nhiên, việc để thông tin kết nối (host, username, password) trong một script không an toàn. Bạn nên dùng Securing Credentials.

Kết nối đến database (3)

- ▶ Ta đã kết nối đến một temporary in-memory database, vốn chưa có dữ liệu.
- ▶ Ta có thể chép dữ liệu cho nó dùng hàm `copy_to()`.
- ▶ Đoạn mã sau chép bảng `nycflights13::flights`

```
copy_to(con, nycflights13::flights, "flights",  
        temporary = FALSE,  
        indexes = list(  
          c("year", "month", "day"),  
          "carrier",  
          "tailnum",  
          "dest"  
        )  
)
```

- ▶ Sau khi chép, ta có thể dùng `tbl()` để tham chiếu đến nó

```
flights_db <- tbl(con, "flights")
```

Kết nối đến database (4)

► In dữ liệu từ `flights_db`

► Kết quả giống một tibble nhưng thật sự nó là một remote source trong một SQLite database.

```
flights_db
```

```
## # Source:   table<flights> [?? x 19]
## # Database: sqlite 3.39.3 [:memory:]
##      year month   day dep_time sched_de~1 dep_d~2 arr_t~3 sched~4 arr_d~5
##      <int> <int> <int>    <int>      <int>    <dbl>    <int>    <int>    <dbl>
##  1  2013     1     1      517        515         2      830      819      19
##  2  2013     1     1      533        529         4      850      830      20
##  3  2013     1     1      542        540         2      923      850      30
##  4  2013     1     1      544        545        -1     1004     1022     -18
##  5  2013     1     1      554        600        -6      812      837     -29
##  6  2013     1     1      554        558        -4      740      728      12
##  7  2013     1     1      555        600        -5      913      854      19
##  8  2013     1     1      557        600         3      700      722
```

Tạo query

- ▶ Để tương tác với database ta thường dùng SQL.
- ▶ Mục đích của dbplyr là để sinh SQL tự động.
 - ▶ Tuy nhiên, SQL là một ngôn ngữ lớn, dbplyr chỉ hỗ trợ một phần và tập trung vào câu lệnh SELECT.

```
flights_db %>% select(year:day, dep_delay, arr_delay)
```

```
## # Source:   SQL [?? x 5]
## # Database: sqlite 3.39.3 [:memory:]
##   year month   day dep_delay arr_delay
##   <int> <int> <int>      <dbl>      <dbl>
## 1  2013     1     1         2         11
## 2  2013     1     1         4         20
## 3  2013     1     1         2         33
## 4  2013     1     1        -1        -18
## 5  2013     1     1        -6        -25
## 6  2013     1     1        -4         12
```

Show query (1)

```
flights_db %>% select(year:day, dep_delay, arr_delay) %>% show_query()
```

```
## <SQL>
```

```
## SELECT `year`, `month`, `day`, `dep_delay`, `arr_delay`
```

```
## FROM `flights`
```

```
flights_db %>% filter(dep_delay > 240) %>% show_query()
```

```
## <SQL>
```

```
## SELECT *
```

```
## FROM `flights`
```

```
## WHERE (`dep_delay` > 240.0)
```

Show query (2)

```
flights_db %>%  
  group_by(dest) %>%  
  summarise(delay = mean(dep_time)) %>%  
  show_query()
```

```
## <SQL>  
## SELECT `dest`, AVG(`dep_time`) AS `delay`  
## FROM `flights`  
## GROUP BY `dest`
```

Show query (3)

```
flights_db %>%  
  group_by(tailnum) %>%  
  summarise(  
    delay = mean(arr_delay),  
    n = n()  
  ) %>%  
  arrange(desc(delay)) %>%  
  filter(n > 100) %>%  
  show_query()
```

```
## <SQL>  
## SELECT *  
## FROM (  
##   SELECT `tailnum`, AVG(`arr_delay`) AS `delay`, COUNT(*) AS `n`  
##   FROM `flights`  
##   GROUP BY `tailnum`  
## )
```

Kết nối đến SQLite database file

```
con <- DBI::dbConnect(RSQLite::SQLite(), dbname = "./company.db")
```

```
DBI::dbListTables(con)
```

```
## [1] "department"      "dependent"        "dept_locations"  "employee"  
## [5] "project"         "works_on"
```

```
DBI::dbListFields(con, "employee")
```

```
## [1] "fname"    "minit"    "lname"    "ssn"      "bdate"    "address"  
## [7] "sex"      "salary"   "superssn" "dno"
```

Tạo SQL query

```
employee <- DBI::dbGetQuery(con, "SELECT * FROM employee")
head(employee)
```

##	fname	minit	lname	ssn	bdate	address	se
## 1	James	E	Borg	888665555	27-11-10	450 Stone, Houston, TX	M
## 2	Franklin	T	Wong	333445555	45-12-18	638 Voss, Houston, TX	M
## 3	John	B	Smith	123456789	55-1-9	731 Fondren, Houston, TX	M
## 4	Jennifer	S	Wallace	987654321	31-6-20	291 Berry, Bellair, TX	M
## 5	Alicia	J	Zelaya	999887777	58-7-19	3321 Castle, Spring, TX	M
## 6	Ramesh	K	Narayan	666884444	52-9-15	975 Fire Oak, Humble, TX	M
##	superssn	dno					
## 1	<NA>	1					
## 2	888665555	5					
## 3	333445555	5					
## 4	888665555	4					
## 5	987654321	4					
## 6	333445555	5					

Các loại phép kết

Các loại phép kết

- ▶ Cú pháp: `xxx_join(x, y, by = "...")`
- ▶ `inner_join()`: tất cả các dòng của x có giá trị **match** với y
- ▶ `left_join()`: `inner_join()` hội với tất cả các dòng của x
- ▶ `right_join()`: `inner_join()` hội với tất cả các dòng của y
- ▶ `full_join()`: tất cả các dòng của x và y
- ▶ `semi_join()`: chiều `inner_join()` trên các cột của x
- ▶ `anti_join()`: `x - semi_join()`

Inner join

- Cho trước $r(X), s(Y)$
- Cú pháp: $r \bowtie s$
- Nghĩa: $r \bowtie s = \{t[X \cup Y] \mid t[X] \in r, t[Y] \in s\}$

Employee

Name	Empld	DeptName
Harry	3415	Finance
Sally	2241	Sales
George	3401	Finance
Harriet	2202	Sales

Dept

DeptName	Manager
Finance	George
Sales	Harriet
Production	Charles

Employee $\bowtie$ Dept

Name	Empld	DeptName	Manager
Harry	3415	Finance	George
Sally	2241	Sales	Harriet
George	3401	Finance	George
Harriet	2202	Sales	Harriet

Left join

- Cho trước $r(X), s(Y)$
- Cú pháp: $r \bowtie s$
- Nghĩa: $r \bowtie s \cup ((r \bowtie s) \times \{(null, ..., null)\})$

$$r \bowtie s \cup \{t[X \cup Y] \mid t[X] \in r \bowtie s, t[Y] = (null, ..., null)\}$$

<i>Employee</i>		
Name	Empld	DeptName
Harry	3415	Finance
Sally	2241	Sales
George	3401	Finance
Harriet	2202	Sales
Tim	1123	Executive

<i>Dept</i>	
DeptName	Manager
Sales	Harriet
Production	Charles

<i>Employee</i> $\bowtie$ <i>Dept</i>			
Name	Empld	DeptName	Manager
Harry	3415	Finance	ω
Sally	2241	Sales	Harriet
George	3401	Finance	ω
Harriet	2202	Sales	Harriet
Tim	1123	Executive	ω

Right join

- Cho trước $r(X), s(Y)$
- Cú pháp: $r \bowtie s$
- Nghĩa: $r \triangleright \triangleleft s \cup \left(\{ (null, \dots, null) \} \times (s \triangleright r) \right)$
 $r \triangleright \triangleleft s \cup \{ t[X \cup Y] \mid t[Y] \in s \triangleright r, t[X] = (null, \dots, null) \}$

<i>Employee</i>		
Name	Empld	DeptName
Harry	3415	Finance
Sally	2241	Sales
George	3401	Finance
Harriet	2202	Sales
Tim	1123	Executive

<i>Dept</i>	
DeptName	Manager
Sales	Harriet
Production	Charles

<i>Employee $\bowtie$ Dept</i>			
Name	Empld	DeptName	Manager
Sally	2241	Sales	Harriet
Harriet	2202	Sales	Harriet
ω	ω	Production	Charles

Full join

- Cho trước $r(X), s(Y)$
- Cú pháp: $r \bowtie s$
- Nghĩa: $(r \bowtie s) \cup (r \ltimes s)$

Employee

Name	EmpId	DeptName
Harry	3415	Finance
Sally	2241	Sales
George	3401	Finance
Harriet	2202	Sales
Tim	1123	Executive

Dept

DeptName	Manager
Sales	Harriet
Production	Charles

Employee ⋈ Dept

Name	EmpId	DeptName	Manager
Harry	3415	Finance	ω
Sally	2241	Sales	Harriet
George	3401	Finance	ω
Harriet	2202	Sales	Harriet
Tim	1123	Executive	ω
ω	ω	Production	Charles

Semi join

- Cho trước $r(X), s(Y)$
- Cú pháp: $r \bowtie s$
- Ngữ nghĩa: $\pi_X(r \supset \triangleleft s)$

Employee

Name	Empld	DeptName
Harry	3415	Finance
Sally	2241	Sales
George	3401	Finance
Harriet	2202	Production

Dept

DeptName	Manager
Sales	Bob
Sales	Thomas
Production	Katie
Production	Mark

Employee $\bowtie$ Dept

Name	Empld	DeptName
Sally	2241	Sales
Harriet	2202	Production

Anti join

- Cho trước $r(X), s(Y)$
- Cú pháp: $r \triangleright s$
- Nghĩa: $r - (r \bowtie s)$

Employee

Name	EmpId	DeptName
Harry	3415	Finance
Sally	2241	Sales
George	3401	Finance
Harriet	2202	Production

Dept

DeptName	Manager
Sales	Sally
Production	Harriet

Employee $\triangleright$ Dept

Name	EmpId	DeptName
Harry	3415	Finance
George	3401	Finance

dplyr vs. SQL

R (dplyr)	SQL
<code>inner_join()</code>	<code>SELECT * FROM x JOIN y ON x.a = y.a</code>
<code>left_join()</code>	<code>SELECT * FROM x LEFT JOIN y ON x.a = y.a</code>
<code>right_join()</code>	<code>SELECT * FROM x RIGHT JOIN y ON x.a = y.a</code>
<code>full_join()</code>	<code>SELECT * FROM x FULL JOIN y ON x.a = y.a</code>
<code>semi_join()</code>	<code>SELECT * FROM x WHERE EXISTS (SELECT 1 FROM y WHERE x.a = y.a)</code>
<code>anti_join()</code>	<code>SELECT * FROM x WHERE NOT EXISTS (SELECT 1 FROM y WHERE x.a = y.a)</code>
<code>intersect(x, y)</code>	<code>SELECT * FROM x INTERSECT SELECT * FROM y</code>
<code>union(x, y)</code>	<code>SELECT * FROM x UNION SELECT * FROM y</code>
<code>setdiff(x, y)</code>	<code>SELECT * FROM x EXCEPT SELECT * FROM y</code>

Tham khảo

1. <https://r4ds.had.co.nz/relational-data.html>
2. <https://github.com/rstudio/cheatsheets/blob/main/data-transformation.pdf>
3. https://en.wikipedia.org/wiki/Relational_algebra
4. <https://dbplyr.tidyverse.org/articles/dbplyr.html>
5. <https://dbplyr.tidyverse.org/articles/translation-verb.html>